

# NAMOSIM: A Robot Motion Planner for Navigation Among Movable Obstacles

Benoit Renault<sup>2 4</sup>, Jacques Saraydaryan<sup>1 3 4</sup>, David Brown<sup>1 4</sup>, and Olivier Simonin<sup>1 2 4</sup>

1 Inria, CHROMA Team 2 INSA Lyon 3 CPE Lyon 4 CITI Laboratory

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- Review [↗](#)
- Repository [↗](#)
- Archive [↗](#)

Editor: [Open Journals](#) [↗](#)

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

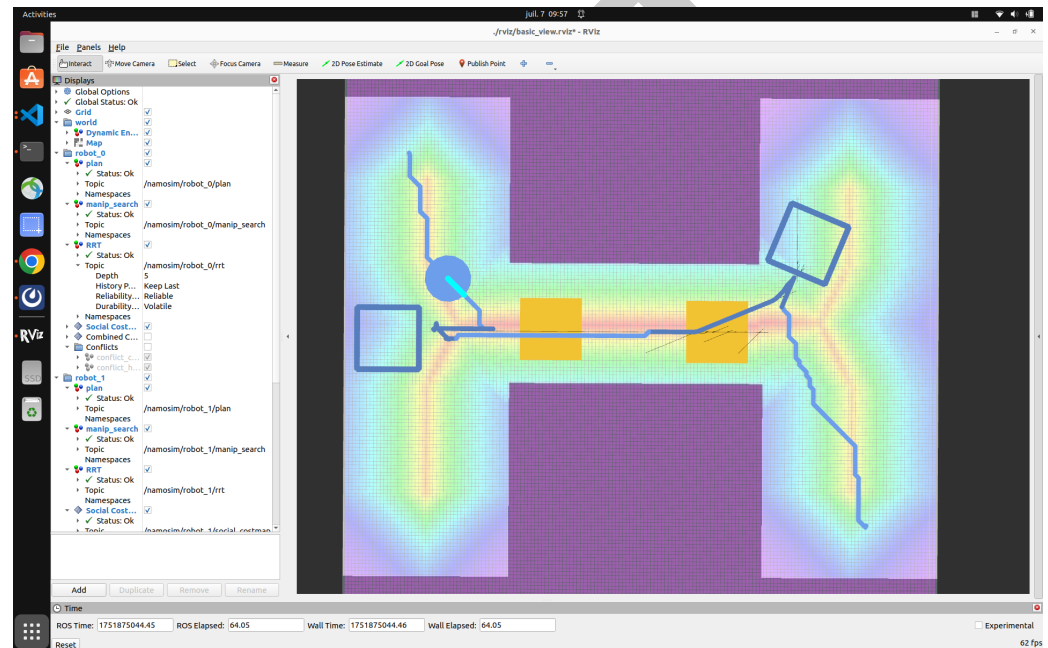


Figure 1: NAMOSIM

## Summary

*NAMOSIM* is a robot motion planning simulator designed for the problem of Navigation Among Movable Obstacles (NAMO). It enables simulation and evaluation of motion planning strategies in 2D environments where certain obstacles can be manipulated by robots to reach their goals. The simulator includes support for holonomic and differential-drive motion models, and integrates with ROS2 for visualization in RViz.

*NAMOSIM* provides a modular agent-based architecture, including a baseline implementation of the Stilman2005 NAMO algorithm. A variety of other agent types are implemented, and new agents utilizing alternative algorithmic approaches can be created and plugged into the planner in a straightforward manner by implementing the **Agent** base class. Thus, new planning strategies can be easily developed, thereby facilitating experimentation with differing approaches, including machine-learning-based agents.

The system utilizes ROS2 messages for visualization of plans using RViz2 and includes a number of custom scenarios to use for testing and benchmarking.

This software is intended for researchers and developers working on robot navigation in dynamic

environments, particularly where physical interaction with the environment is necessary.

## Statement of need

Most navigation planners assume static environments and non-interactive environments, limiting their usefulness in complex real-world applications. NAMO problems involve not only path planning but also introduce the need for reasoning about which obstacles to move, where to move them, and how to combine standard navigation with obstacle manipulation. NAMOSIM addresses this by offering a simulation environment explicitly designed to study and prototype NAMO algorithms. NAMOSIM additionally supports multi-robot environments and thus facilitates reproducible research in multi-robot navigation among movable obstacles (MR-NAMO).

## Major Features

NAMOSIM provides a robust set of features to support research and development in Navigation Among Movable Obstacles (NAMO):

- **Modular Agent-Based Architecture:** The simulator is built around a flexible Agent interface, allowing users to implement and test custom NAMO planning algorithms. A baseline implementation of the Stilman2005 planner is included for immediate use and benchmarking.
- **Support for Multiple Robot Models:** NAMOSIM supports both holonomic and differential drive robot models, enabling realistic simulation of various robotic platforms.
- **ROS2 Integration:** Full compatibility with ROS2 allows seamless deployment on both simulated and physical robots, with built-in support for visualization in RViz2 and integration with ROS2 navigation stacks.
- **2D Environment Simulation:** The simulator provides a customizable 2D environment where users can define static and movable obstacles, supporting complex scenarios for testing navigation and manipulation strategies.
- **Extensive Testing and Documentation Utilities:** NAMOSIM includes tools for automated testing of planning algorithms and generating comprehensive documentation, facilitating reproducible research and educational use.
- **Multi-Robot Coordination:** The simulator supports multi-robot scenarios, enabling the study of implicit coordination and conflict resolution in NAMO tasks, as explored in related research (Renault et al., 2024).

These features make NAMOSIM a versatile tool for prototyping, evaluating, and deploying NAMO algorithms in diverse robotic applications.

## Customizable Scenarios

NAMOSIM environments, or **scenarios**, are stored in SVG format and can be edited using any SVG editor such as Inkscape. The scenario SVG file contains the following key elements:

- The geometry of the static map
- The polygons and orientations of all robots and movable obstacles
- Configuration settings that define the behavior of the environment and robots.

The static map can also be included as an image layer inside the SVG to conveniently include ROS grid-map images generated by standard mapping tools.

## 62 Architecture

63 At a high-level, NAMOSIM executes a SENSE-THINK-ACT loop that performs the following  
64 functions at each iteration:

- 65 1. SENSE: Each agent senses the environment and updates its internal representation of it.
- 66 2. THINK: Each agent computes a new plan or updates its current plan.
- 67 3. ACT: Each agent selects a single discrete action to execute.

68 The loop is expected to execute at a regular frequency with the assumption that all agent  
69 functions are synchronized at run sequentially.

## 70 Collision Detection

71 Custom agents are free to implement their own collision detection, however our baseline  
72 Stilman2005 agent detects collisions using a simple binary-occupancy grid when the robot  
73 footprint is circular. However, when transporting a movable obstacle, the robot footprint is  
74 non-circular, and collision detection is based on the convex-swept-volume of the combined  
75 robot-obstacle footprint's motion to guarantee all possible collisions are detected.

## 76 Conflict Avoidance and Deadlock Resolution

77 The baseline Stilman2005 agent has the capability to avoid conflicts and attempt to resolve  
78 deadlocks. Conflict avoidance works by looking ahead along the agent's current plan for a  
79 fixed number of steps, called the **conflict horizon**. Within the horizon, the agent simulates  
80 each planned action and checks for a number of possible conflicts. For example, the agent  
81 may have planned to move a certain obstacle which has been moved by another robot and  
82 is no longer at the expected location. Or as another example, an action within the conflict  
83 horizon may collide with another robot that currently crossing the planned path.

84 The Stilman2005 agent avoids conflicts by either pausing or planning around them. A  
85 deadlock is detected when a given conflict configuration is re-detected multiple times, even  
86 after replanning. To resolve deadlocks, the agent follows an evasion strategy as described in  
87 our IROS-2024 paper [1].

## 88 The Core NAMO Algorithm

89 The core NAMO algorithm implemented in our Stilman2005 agent, is based on the idea of  
90 moving obstacles in order to merge disjoint components of the robot's configuration space.  
91 The map is divided into a set of disjoint connected-components where each cell in a given  
92 component is reachable from all the other cells in the same component. The connected-  
93 components must be separated from each other by movable obstacles, otherwise they are  
94 unreachable. The agent's goal is to move obstacles in order to join different components to  
95 open a path to the goal.

96 The algorithm computes a plan by recursively performing the following two stages:

- 97 1. **SELECT\_OBSTACLE\_AND\_COMPONENT**: The first stage performs a simplified  
98 A\* grid search where the agent is allowed to pass through movable obstacles. It returns  
99 the ID of the first movable obstacle encountered on the path to the goal and the ID of  
100 the component encountered after passing through the obstacle.
- 101 2. **OBSTACLE\_MANIPULATION\_SEARCH**: The second stage first finds a **transit path**  
102 from the robot's current position to a grasp pose near the obstacle. Then it finds  
103 a **transfer path** by doing an obstacle manipulation search to join the robot's current  
104 component to the component selected in stage 1. If this stage fails for any reason, the  
105 obstacle and component pair are added to an avoid-list and the algorithm goes back to  
106 stage 1.

107 This algorithm is explained in full detail in [3].

## 108 Acknowledgements

109 This research was supported by an Inria ADT initiative. We express our gratitude to Benoit  
110 Renault, whose PhD thesis forms the foundation of this work.

## 111 References

- 112 Renault, B., Saraydaryan, J., Brown, D., & Simonin, O. (2024). Multi-Robot Navigation among  
113 Movable Obstacles: Implicit Coordination to Deal with Conflicts and Deadlocks. *IEEE/RSJ*  
114 *International Conference on Intelligent Robots and Systems (IROS)*. [https://hal.science/hal-](https://hal.science/hal-04705395)  
115 [04705395](https://hal.science/hal-04705395)
- 116 Renault, B., Saraydaryan, J., & Simonin, O. (2020). Modeling a Social Placement Cost to  
117 Extend Navigation Among Movable Obstacles (NAMO) Algorithms. *IEEE/RSJ IROS 2020*.  
118 <https://doi.org/10.1109/IROS45743.2020.9340892>
- 119 Benoit Renault. NAvigation en milieu MODifiable (NAMO) étendue à des contraintes sociales  
120 et multi-robots. Robotique [cs.RO]. INSA de Lyon, 2023. Français. NNT : 2023ISAL0105 .  
121 tel-04418723v2